Lab no 03: Simulate Flip-Flop & 4-Bit Johnson Counter

The purpose of this Lab is to learn to:

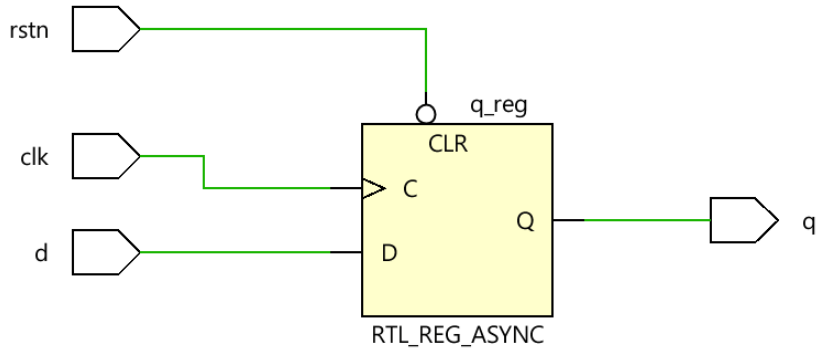
1) Simulate D Flip-Flop on ModelSim and verify the D Flip-Flop function using testbench. In this lab, you will build the D Flip-Flop using the behavioral description.
2) Simulate a 4-Bit Johnson Counter. You will write the structure description of the counter in terms of flip-flops.

   Objective: Simulate sequential circuits.

Parts: -

1. Simulate D Flip-Flop.
2. Simulate a 4-Bit Johnson Counter.

Part 1. Simulate Asynchronous Reset D Flip-Flop.



D-Flip-Flop Schematic https://www.chipverify.com/verilog/verilog-d-flip-flop-async-reset

A D flip-flop (sometimes called a "data flip-flop") is a type of sequential digital circuit that stores a single bit of data. It is called a "flip-flop" because it can store a single bit of information and "flip" between two states. The D flip-flop output q follows the input d at the given edge of a clock.

The D flip-flop can be useful in many digital circuit applications, such as in synchronous circuits where data is transferred between different parts of the circuit on each clock cycle. It can also be used in counters, shift registers, and other sequential logic circuits.

Behavioral Verilog Code for D-FlipFlop

d_ff.v

```
// ----------------- D-Flip Flop --------------------//
// asynchronous reset D-flipflop
module d_ff ( d, rstn, clk, q) ;
input d, rstn, clk;
output reg q;
// D-flipflop is sensitive to positive edge clock and negative edge reset
always @ (posedge clk or negedge rstn)
// the output q is zero at negative reset
      if (!rstn)
           q <= 0;
// else the output q follows the d input
       else
           q <= d;
endmodule
```

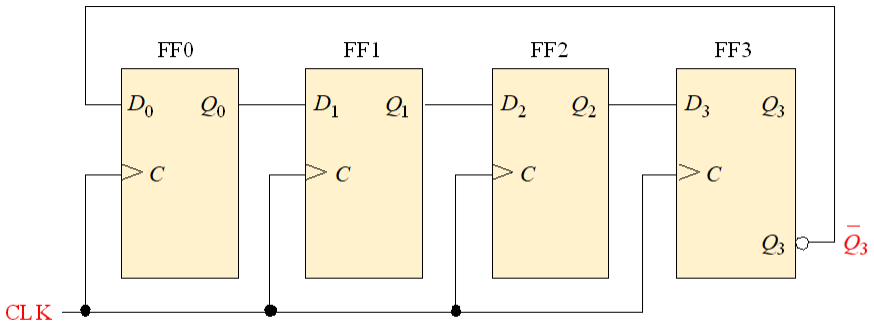
```
// ------------------------------------------------//
```

Part 2. Simulate 4-Bit Johnson Counter.

A Johnson counter is a type of sequential digital circuit that produces a sequence of binary values using a ring of flip-flops. It is also known as a "twisted ring counter" or a "walking ring counter."

Johnson counters have several applications in digital circuits, such as in frequency division, shift register operations, and waveform generation. They can also be used as a simple pseudo-random number generator.

A Johnson counter can be implemented using D flip-flops or JK flip-flops. In a D flip-flop implementation, the Q output of each flip-flop is connected to the D input of the next flip-flop in the ring. The output of the last flip-flop in the ring is fed back to the input of the first flip-flop.



CLK	Q_0	Q_1	Q_2	Q_3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1
5	0	1	1	1
6	0	0	1	1
7	0	0	0	1

Structural Verilog Code for 4-Bit Johnson Counter

JohnsonCounter.v

```
// ------------ 4-Bit Johnson Counter ---------------//
module JohnsonCounter(d, rstn, clk, q);
// reset and clock are one-bit inputs
input rstn, clk;
// d is a three-bit input
input [3:0] d;
// q is a three-bit output
output [3:0] q;
// define a new wire not_q3 to invert the last bit of the q output
wire not_q3;
```

```
// invert the last bit q is a three-bit output
assign not_q3 = ~q[3];
// build the structure of the 4-bit Johnson Counter using four D flip-flops.
d_ff FF0 (not_q3, rstn, clk, q[0]) ;
d_ff FF1 (q[0], rstn, clk, q[1]) ;
d_ff FF2 (q[1], rstn, clk, q[2]) ;
d_ff FF3 (q[2], rstn, clk, q[3]) ;
endmodule
// ------------------------------------------------//
```

Test Using Do Files

A do file in ModelSim is a text file that contains a list of commands that can be executed in the ModelSim simulation environment. It is used to automate simulation tasks and to perform simulations without having to manually enter commands each time.

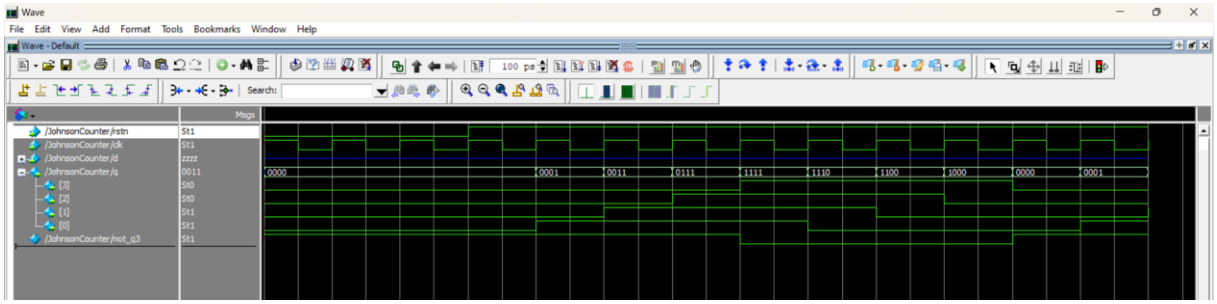
To execute a do file in ModelSim, you can use the "do" command in the ModelSim console window, followed by the path to the do file. For example: do filename.do

- Save the following commands in a text file named JohnsonCounter.do
- In the transcript window, **write** do JohnsonCounter.do

```
vlib work
vmap work work
vlog *.v
vsim JohnsonCounter
add wave sim:/JohnsonCounter/*
force -freeze sim:/JohnsonCounter/rstn 0 0
force -freeze sim:/JohnsonCounter/clk 1 0, 0 {50 ps} -r 100
run
```

```
run
run
force -freeze sim:/JohnsonCounter/rstn 1 0
run
run
run
run
run
```

Check the output in the Wave Window



Task:

Write a testbench to verify the function of the 4-bit Johnson Counter.

Review Lab 02, you wrote a testbench for a 2-to-1 multiplexer.